Informe Técnico

Práctica 2 — Programación Orientada a Objetos

Framework de regresión lineal múltiple en C++

Curso: Lenguajes y Paradigmas de Programación

Universidad: EAFIT — Ingeniería de Sistemas

Periodo: 2026-2

Fecha de entrega: 12/11/2026

Nombre: Daniel Alejandro Tabares Vasquez

ID / Cédula: 1000344311 / 1018230240

Correo institucional: databaresv@eafit.edu.co

1. Video de sustentación y repositorio

Aquí dejo el enlace al video, donde explico y sustento el código, y el enlace al repositorio donde está el código fuente.

- Video: _____
- Repositorio: _____

2. Documentación técnica

2.1 Cómo diseñamos la clase LinearRegression

La idea central fue tener una sola clase, LinearRegression, que guardara todo lo necesario para un modelo de regresión lineal múltiple entrenado con gradiente descendente. No usé librerías externas, así que el escalado, el entrenamiento, la predicción y las métricas de error los programé yo mismo con ciclos sobre vector<double> y vector<vector<double>>. El modelo calcula $\hat{y}$ = bias + $w_1 \cdot x_1$ + … + w_d·x_d y busca minimizar el error cuadrático medio (MSE).

Una ventaja que me gustó de este diseño es que la misma clase funciona para regresión simple (una variable) y para regresión múltiple (varias variables) sin cambiar nada, porque el número de pesos se define en el momento de entrenar según cuántas columnas tenga X.

Dejé los atributos privados y los mostré solo a través de getters, ya que así nadie puede cambiar el estado del modelo por fuera (encapsulamiento). El enunciado pedía como mínimo weights y bias, y yo agregué los demás porque los necesitaba para entrenar y escalar.

Como el enunciado prohíbe librerías externas, las operaciones de estadística y de matrices las escribí a mano dentro del espacio de nombres MathUtils (media, desviación estándar, extracción de columnas, transpuesta y producto matriz-vector). Solo usé librerías estándar de C++, que son vector, cmath (únicamente para sqrt), fstream, sstream, string y stdexcept. Para no escribir tanto, en el código definí los alias Vec para vector<double> y Matrix para vector<vector<double>>.

Atributo	Tipo	Para qué sirve
weights	vector<double>	Pesos del modelo, uno por cada variable de entrada. Arrancan en cero cuando se llama a fit().
bias	double	El término independiente (intercepto) del modelo.
learningRate	double	La tasa de aprendizaje del gradiente descendente.
epochs	int	Cuántas iteraciones (épocas) hace el entrenamiento.
means	vector<double>	La media de cada columna de X, calculada con los datos de entrenamiento. La guardamos para escalar después los datos de prueba con los mismos valores.
stds	vector<double>	La desviación estándar de cada columna de X, también calculada con los datos de entrenamiento.

2.2 Utilidades matemáticas (MathUtils)

Estas son las funciones auxiliares que hice sin librerías externas y que usan los métodos de LinearRegression.

Función	Qué hace
double mean(const Vec& v)	Calcula la media aritmética, es decir, suma los elementos y los divide entre la cantidad.
double stdDev(const Vec& v, double m)	Calcula la desviación estándar poblacional a partir de la media m, sacando la raíz de la suma de $(x_i - m)^2$ dividida por n.
Vec column(const Matrix& M, size_t j)	Saca la columna j de una matriz y la devuelve como vector.
Matrix transpose(const Matrix& M)	Recibe una matriz n×d y devuelve su transpuesta d×n.
Vec matVec(const Matrix& M, const Vec& v)	Multiplica una matriz por un vector y devuelve r, donde $r_i = \Sigma M_i \cdot v$.
void checkMatrix(const Matrix& X, const string& contexto)	Revisa que X no esté vacía, que tenga columnas y que todas las filas midan lo mismo. Si algo falla, lanza invalid_argument con un mensaje que dice en qué método y en qué fila está el problema.

2.3 Métodos de la clase

LinearRegression(double lr = 0.01, int epochs = 1000). Es el constructor. Recibe la tasa de aprendizaje y el número de épocas (si no se le pasan, usa esos valores por defecto) y deja el bias en 0. Si lr o epochs no son mayores que 0, lanza invalid_argument. Los vectores weights, means y stds quedan vacíos hasta que se llamen fit() y data_scaling().

void data_scaling(Matrix& X, bool fitScaler = true). Estandariza cada columna de X con la fórmula z = (x − media) / desviación, apoyándose en MathUtils::mean y MathUtils::stdDev. Como recibe la matriz por referencia, la modifica directamente. Cuando fitScaler = true, calcula y guarda means y stds, y por eso lo uso con los datos de entrenamiento. Cuando fitScaler = false, reutiliza los valores que ya tenía guardados, y lo uso con los datos de prueba para que queden escalados igual que el entrenamiento y no se filtre información del test. Si una columna tiene desviación 0, la cambio por 1 para no dividir por cero. Lanza logic_error si se intenta escalar el test sin haber escalado antes el entrenamiento, e invalid_argument si la matriz está vacía o si no coincide el número de columnas.

void fit(const Matrix& X, const Vec& y). Entrena el modelo con gradiente descendente por lotes, y espera que X ya venga escalada. Primero valida que X no esté vacía y que X y y tengan el mismo número de filas. Después calcula una sola vez la transpuesta de X, pone weights en cero y en cada época hace cuatro cosas: (1) predice con matVec(X, weights) y le suma el bias; (2) calcula el error $err_i = \hat{y}_i - y_i$; (3)

saca los gradientes dw = (2/n)·X^T·err y dbias = (2/n)·Σ err_i; y (4) actualiza w ← w − lr·dw y bias ← bias − lr·dbias.

Vec predict(const Matrix& X) const. Devuelve un vector con la predicción de cada fila de X (ya escalada), que se calcula como matVec(X, weights) más el bias. Es un método const porque no cambia nada del modelo. Lanza logic_error si el modelo todavía no se ha entrenado, e invalid_argument si X está vacía o si su número de columnas no es igual al número de pesos.

double score(const Matrix& X, const Vec& y) const. Mide el error de las predicciones con el error cuadrático medio, MSE = (1/n)·Σ$(\hat{y}_i - y_i)^2$, así que entre más bajo, mejor es el ajuste. Este es el método de error que pide el enunciado. Lanza invalid_argument si la cantidad de valores reales no coincide con la de predicciones.

double r2_score(const Matrix& X, const Vec& y) const. Es un método extra que agregué para calcular el coeficiente de determinación $R^2 = 1 - SSE/SST$, donde SSE es la suma de errores cuadráticos del modelo y SST es la suma de cuadrados respecto a la media de y. Vale 1 cuando el ajuste es perfecto y 0 cuando el modelo no hace nada mejor que predecir siempre la media. Si SST da 0, devuelve 0, y valida los tamaños igual que score.

getWeights() / getBias() / getMeans() / getStds(). Son los métodos de consulta que devuelven los pesos, el bias, las medias y las desviaciones guardadas. Con getWeights() y getBias() cumplo la prueba que sugiere el enunciado.

const double* getWeightsAddress() const. Devuelve la dirección del buffer interno de weights (weights.data()). Lo uso en el análisis de memoria para mostrar que ese buffer está en el heap, incluso cuando el objeto está en memoria global o en el stack.

2.4 Funciones auxiliares y de prueba

double parseNumber(const string& cell, int numLinea). Pasa un texto a número con stod y revisa que no quede texto sobrando (por ejemplo "8abc"). Si falla, lanza runtime_error diciendo en qué línea está y cuál fue el valor problemático.

void loadCSV(istream& in, Matrix& X, Vec& y) / void loadCSV(const string& filename, Matrix& X, Vec& y). Leen los datos desde un flujo o desde un archivo. Se saltan el encabezado, toman la última columna como salida y y las anteriores como entradas X. Detectan solas el separador (;, coma o tabulador) y aceptan decimales con coma cuando el separador es ;. Además, validan que cada fila tenga al menos 2 columnas, que todas midan lo mismo y que el archivo sí tenga datos, y si algo sale mal lanzan runtime_error con el número de línea (o avisan que no pudieron abrir el archivo). Hice la versión por flujo para poder probar la lectura sin tener que crear archivos.

void splitData(X, y, int k, Xtr, ytr, Xte, yte). Separa los datos en entrenamiento y prueba, mandando cada k-ésima fila al conjunto de prueba (con k = 5, o sea el 20 %) y el resto al de entrenamiento. Preferí esto a tomar simplemente las últimas filas, porque el archivo de helados viene ordenado por temperatura y una división en orden habría sesgado la prueba. Valida que k ≥ 2 y que X y y tengan el mismo número de filas.

pruebaUnitaria() / pruebaHelados() / pruebaEstudiantes() / pruebaMemoria() / pruebaValidaciones(). Son las funciones de prueba. pruebaUnitaria() comprueba data_scaling, fit, predict y score con datos

cuyo resultado ya conozco. pruebaHelados() corre la regresión simple y además una variante con la variable x^2 agregada. pruebaEstudiantes() corre la regresión múltiple. pruebaMemoria() crea modelos en memoria global, local y dinámica, un vector y un arreglo dinámico, e imprime sus direcciones y tamaños. Por último, pruebaValidaciones() provoca nueve errores de uso a propósito para mostrar que el programa los detecta y avisa.

imprimirModelo() / caso() / ejecutar() / main(). imprimirModelo() muestra la dirección, el tamaño y el buffer de pesos de un modelo. caso() ejecuta una prueba de validación y muestra el mensaje del error controlado. ejecutar() corre una prueba atrapando cualquier excepción, para que si una sección falla no se detenga todo lo demás. Y main() ejecuta las cinco pruebas en orden.

3. Ejemplos de ejecución

Todas las salidas que muestro aquí salieron de compilar y correr el código en OnlineGDB (C++). Los datos los leí de ice_cream.txt y student_scores.txt, que son copias de los CSV de EAFIT Interactiva pero con extensión .txt, porque OnlineGDB trata los .csv como scripts de enlazado.

3.1 Pruebas unitarias de los métodos

Para probar cada método por separado usé datos cuyo resultado ya sabía. A data_scaling le pasé la matriz A = {{1,10},{2,20},{3,30},{4,40}}. A fit le pasé X = {{1},{2},{3},{4}} y y = {3,5,7,9}, que cumplen exactamente y = 2x + 1, con lr = 0.01 y 1000 épocas. Para predict y score usé X_test = {{5},{6}}, primero con y_test = {11,13} (valores exactos) y luego con y_test = {10,14} (valores con error).

```
=== Prueba data_scaling ===
-1.34164 -1.34164
-0.447214 -0.447214
0.447214 0.447214
1.34164 1.34164

=== Prueba fit (y = 2x + 1) ===
weights: 2.23607
bias: 6

=== Prueba predict y score ===
y_hat: 11
y_hat: 13
MSE (y_test exacto): 4.10695e-16
R2 (y_test exacto): 1
MSE (y_test con error): 1
R2 (y_test con error): 0.75
```

Prueba	Resultado	Cómo lo interpreto
data_scaling (columna 1,2,3,4)	−1.34164 … 1.34164	La media es 2.5 y la desviación 1.118, así que (1 − 2.5)/1.118 = −1.34164. La segunda columna queda idéntica porque es la primera multiplicada por 10.
fit (y = 2x + 1)	peso 2.23607; bias 6	El modelo aprende en la escala estandarizada, por eso el peso es 2·1.118 = 2.236 y el bias es 2·2.5 + 1 = 6, que es el valor de y en la media de x.

Prueba	Resultado	Cómo lo interpreto
predict (x = 5, 6)	11 y 13	Coincide con y = 2x + 1, o sea que el modelo reproduce la relación exacta y generaliza bien a datos que no había visto.
score con y exacto	MSE ≈ 4.1e-16	Es prácticamente cero; ese pequeño residuo es solo error numérico de punto flotante.
score con y con error	MSE = 1; R^2 = 0.75	Los errores de +1 y −1 dan MSE = (1+1)/2 = 1. Con SST = 8 y SSE = 2, el R^2 queda en 1 − 2/8 = 0.75.

3.2 Regresión simple con ice_cream.txt

Este archivo tiene 49 filas con la temperatura en °C y las ventas de helados. Separé el 20 % para prueba, lo que dejó 39 filas para entrenar y 10 para probar, y entrené con lr = 0.1 y 2000 épocas.

```
=== Regresion simple: ice_cream.txt ===
Filas leidas: 49
Train: 39 Test: 10
weights: -1.07339
bias: 15.9878
y_hat: 18.0246 y_real: 41.843
y_hat: 17.5343 y_real: 30.0011
y_hat: 17.2074 y_real: 13.2758
y_hat: 16.6689 y_real: 10.9577
y_hat: 16.1445 y_real: 0.328626
y_hat: 15.848 y_real: 4.62569
y_hat: 15.4227 y_real: 3.78257
y_hat: 15.1252 y_real: 10.3366
y_hat: 14.8247 y_real: 22.8394
y_hat: 14.6245 y_real: 17.844
MSE test: 137.992
R2 test: 0.0809659

=== Extra: regresion con [x, x^2] ===
weights: -2.23798 11.7628
bias: 15.9878
MSE test: 17.3678
R2 test: 0.884329
```

Lo primero que noté fue que la regresión lineal simple sacó un R^2 de apenas 0.08, y que sus predicciones eran casi constantes (entre 14 y 18). Revisando los datos entendí por qué, ya que en este archivo la relación entre temperatura y ventas no es una línea sino una parábola en forma de U. Las ventas son altas cuando hace mucho frío, bajan al mínimo cerca de 0 °C y vuelven a subir cuando hace calor, y por eso la correlación lineal es de solo −0.18.

Entonces probé agregar x^2 como segunda columna, y con la misma clase LinearRegression el R^2 subió a 0.88 y el MSE bajó de 137.99 a 17.37. El modelo sigue siendo lineal en los pesos, aunque una de las variables no lo sea.

3.3 Regresión múltiple con student_scores.txt

Este archivo tiene 200 filas con cuatro variables de entrada (horas_estudiadas, horas_de_sueño, porcentaje_de_asistencia y calificaciones_previas) y la nota del examen como salida. Usé 160 filas para entrenar y 40 para probar, con lr = 0.1 y 2000 épocas. Abajo están las primeras 10 predicciones del conjunto de prueba.

```
=== Regresion multiple: student_scores.txt ===
Filas leidas: 200 Variables: 4
Train: 160 Test: 40
weights: 4.98264 1.16502 1.58924 2.65256
bias: 33.8669
y_hat: 34.1413 y_real: 30.2
y_hat: 35.3962 y_real: 35.7
y_hat: 32.5966 y_real: 29.3
y_hat: 29.7239 y_real: 34.1
y_hat: 43.5868 y_real: 46
y_hat: 29.538 y_real: 30
y_hat: 35.3941 y_real: 34.5
y_hat: 33.873 y_real: 32
y_hat: 36.0187 y_real: 34.2
y_hat: 27.7485 y_real: 31.8
MSE test: 7.44703
R2 test: 0.848325
```

Variable (peso)	Valor	Cómo lo interpreto
horas_estudiadas	4.98264	Es la variable que más influye, porque si las horas de estudio suben una desviación estándar, la nota esperada sube casi 5 puntos.
calificaciones_previas	2.65256	Es la segunda en importancia.
porcentaje_de_asistencia	1.58924	Tiene una influencia moderada.
horas_de_sueño	1.16502	Es la que menos influye de las cuatro.
bias	33.8669	Es la nota que esperaríamos para un estudiante promedio en todas las variables.
R^2 / MSE (test)	0.848 / 7.447	El modelo explica cerca del 85 % de la variación de la nota en datos que no vio al entrenar.

Como los pesos están en escala estandarizada, se pueden comparar entre sí. Además, el orden de importancia que salió coincide con las correlaciones de cada variable con la nota (horas estudiadas 0.78, calificaciones previas 0.43, asistencia 0.23 y sueño 0.19), lo que me dio confianza en el resultado.

3.4 Pruebas de validación (manejo de errores)

En pruebaValidaciones() provoqué a propósito nueve errores de uso. En todos, el programa detecta el problema, lanza una excepción con un mensaje claro y sigue funcionando, en vez de cerrarse de golpe. Esto lo agregué porque, antes de tener estas validaciones, varios de estos casos terminaban en un error de segmentación o en una excepción de stod que nadie atrapaba.

Caso que provoqué	Mensaje del programa
1. fit con X vacía	fit: la matriz X esta vacia
2. escalar el test sin escalar antes el entrenamiento	data_scaling: primero debe escalar los datos de entrenamiento (fitScaler = true)
3. predict sin entrenar	predict: el modelo no ha sido entrenado, llame a fit() primero
4. predict con distinto número de columnas	predict: X tiene 1 columnas y el modelo se entreno con 2
5. score con y de distinto tamaño	score: hay 3 predicciones y 5 valores reales
6. archivo con texto en lugar de números	linea 2: 'x' no es un numero
7. archivo con filas de distinto largo	linea 3: tiene 2 columnas y se esperaban 3
8. tasa de aprendizaje inválida	constructor: la tasa de aprendizaje debe ser mayor que 0
9. archivo inexistente	no se pudo abrir el archivo no_existe.txt

4. Análisis de memoria

4.1 Objetos en memoria global, local y dinámica

Creé tres objetos LinearRegression, uno en cada tipo de memoria. El global lo declaré por fuera de todas las funciones, el local lo declaré dentro de una función (así queda en el stack) y el dinámico lo creé con new (así queda en el heap). A los tres los entrené con datos de ejemplo para que su buffer de pesos tuviera algo adentro. Hay que tener en cuenta que las direcciones cambian cada vez que se ejecuta el programa, porque el sistema operativo aleatoriza el espacio de direcciones, así que lo que importa es el patrón y no los números exactos. Esta fue la salida en OnlineGDB:

```
=== Analisis de memoria ===
[Global]
  direccion del objeto : 0x64e7c268c3a0
  sizeof(objeto)       : 96 bytes
  buffer de weights    : 0x64e7dfae1d50
[Local - stack]
  direccion del objeto : 0x7ffd5ddfe720
  sizeof(objeto)       : 96 bytes
  buffer de weights    : 0x64e7dfae2130
[Dinamico - heap]
  direccion del objeto : 0x64e7dfae2500
  sizeof(objeto)       : 96 bytes
  buffer de weights    : 0x64e7dfae2660
  (la variable puntero 'modeloDinamico' vive en el stack: 0x7ffd5ddfe6f8, y apunta a
0x64e7dfae2500)
```

Objeto	Dirección	sizeof	Dónde vive y por qué
Global	0x64e7c268c3a0	96 bytes	Está en el segmento de datos del programa. Se crea antes de main() y dura toda la ejecución.
Local	0x7ffd5ddfe720	96 bytes	Está en el stack, que se reconoce por la dirección muy alta (0x7ff…). Se destruye solo cuando termina la función.
Dinámico	0x64e7dfae2500	96 bytes	Está en el heap (la zona 0x64e7df…, justo al lado de los buffers). Lo crea new y existe hasta que se llama delete. El puntero que lo referencia ocupa 8 bytes y está en el stack, en 0x7ffd5ddfe6f8.

Los tres objetos miden lo mismo porque sizeof depende de cómo está definida la clase y no de dónde se crea el objeto.

Otra cosa que me llamó la atención es que el buffer interno de weights siempre aparece en direcciones del heap (0x64e7df…), aunque el objeto sea global o local. Esto pasa porque std::vector reserva dinámicamente la memoria para sus elementos, y por eso sizeof no cuenta esos datos sino solo el "encabezado" del vector.

4.2 Por qué el tamaño es 96 bytes

Miembro	Bytes	Observación
weights (vector<double>)	24	Son tres punteros de 8 bytes (inicio, fin y capacidad).
bias (double)	8	
learningRate (double)	8	
epochs (int)	4	
relleno (padding)	4	Se agrega porque el siguiente miembro, que es un vector, tiene que empezar en una dirección múltiplo de 8.
means (vector<double>)	24	
stds (vector<double>)	24	
Total	96	Sin relleno la suma daría 3·24 + 2·8 + 4 = 92, y el relleno la sube a 96, que sí es múltiplo de 8.

El propio programa imprime el tamaño de cada pieza, y esa salida confirma la cuenta de la tabla:

```
--- Tamanos de los componentes ---
sizeof(double)                 : 8
sizeof(int)                    : 4
sizeof(vector<double>)         : 24
sizeof(LinearRegression)       : 96
```

```
sizeof(LinearRegression*)      : 8
Suma sin padding: 3*24 + 2*8 + 4 = 92
```

4.3 Vector y arreglo de LinearRegression en memoria dinámica

El vector lo creé con new vector<LinearRegression>(5, LinearRegression(0.1, 100)), y esta fue la salida:

```
--- Vector de LinearRegression en el heap ---
N                              : 5
sizeof(vector<LinearRegression>)  : 24 bytes (solo el 'encabezado')
Bytes de los elementos en el heap : 480 (5 x 96)
Direccion del objeto vector       : 0x64e7dfae2680
Direccion del primer elemento     : 0x64e7dfae26a0
  elemento 0 en 0x64e7dfae26a0
  elemento 1 en 0x64e7dfae2700
  elemento 2 en 0x64e7dfae2760
  elemento 3 en 0x64e7dfae27c0
  elemento 4 en 0x64e7dfae2820
Distancia entre elementos         : 96 bytes
```

El objeto vector mide 24 bytes, igual que cualquier vector<double>, porque lo único que guarda son tres punteros, y ese tamaño no cambia aunque tenga 5 o 5000 elementos. Los elementos, en cambio, ocupan 5 × 96 = 480 bytes seguidos en el heap, separados exactamente 96 bytes entre sí, como se ve en las direcciones (…6a0, …700, …760, …7c0, …820). A su vez, cada uno de esos elementos tiene sus propios buffers de weights, means y stds en otros bloques del heap. Al final del programa liberé la memoria con delete vec y delete modeloDinamico.

Como el enunciado también se puede entender como un arreglo dinámico clásico, creé además new LinearRegression[3] con el constructor por defecto. Esta fue la salida, de la misma ejecución:

```
--- Arreglo dinamico new LinearRegression[3] ---
Bytes del bloque en el heap       : 288 (3 x 96)
sizeof(puntero arr)               : 8 bytes (no conoce el tamano del bloque)
Direccion del bloque              : 0x64e7dfae2898
  arr[0] en 0x64e7dfae2898
  arr[1] en 0x64e7dfae28f8
  arr[2] en 0x64e7dfae2958
Distancia entre elementos         : 96 bytes
```

El bloque ocupa 3 × 96 = 288 bytes seguidos en el heap, con los elementos separados 96 bytes. La diferencia con el vector es que aquí no hay un objeto de 24 bytes con su propio encabezado, sino solo un puntero de 8 bytes (sizeof(arr)) que apunta al primer elemento y no sabe cuántos elementos hay. Por lo tanto, soy yo quien tiene que acordarse de que son 3 y liberarlos con delete[] (no con delete).

4.4 Fallos históricos por gestión de memoria dinámica

Investigué tres casos de gran impacto que tienen su origen en el manejo de memoria dinámica. Dos de ellos son lecturas fuera de los límites de un buffer (Heartbleed y Cloudbleed) y el otro es un uso de memoria después de haberla liberado (Operación Aurora). En cada caso cuento qué pasó, quiénes estuvieron implicados, qué consecuencias tuvo, cómo se pudo arreglar y de dónde saqué la información.

Caso 1. Heartbleed (OpenSSL): la fuga de memoria que dejó expuestas claves privadas

Qué pasó. En abril de 2014 se hizo pública una vulnerabilidad crítica en OpenSSL, la librería que protege las conexiones HTTPS de gran parte de internet. El fallo, registrado como CVE-2014-0160, le permitía a un atacante leer la memoria de un servidor sin tener que autenticarse. Se conoció públicamente el 7 de abril de 2014, después de que Google Security y Codenomicon lo descubrieran cada uno por su lado. Para dimensionarlo, según OWASP, Apache (que usa OpenSSL para HTTPS) estaba en el 66 % de los sitios web de acuerdo con Netcraft.

Por qué falló. El error estaba en la extensión heartbeat de TLS, que sirve para comprobar que una conexión sigue viva. Antes de una llamada a memcpy() faltaba revisar los límites, y esa llamada usaba como longitud un dato que mandaba el cliente y que nadie validaba. Así, el atacante enviaba un mensaje pequeño pero decía que era mucho más largo, y el servidor copiaba más bytes de los que realmente había en el buffer. El NIST lo clasifica como una lectura fuera de límites en el heap, es decir, en memoria reservada dinámicamente.

Consecuencias. Con cada petición, el servidor devolvía hasta 64 KB de su propia memoria. Ahí podían aparecer las claves privadas del servidor, además de contraseñas y datos de sesión de los usuarios. Como el contenido del heap va cambiando, el atacante solo tenía que repetir la petición muchas veces para ir juntando más datos.

Personas y empresas implicadas. El proyecto OpenSSL, porque el error estaba en su código; los investigadores de Google Security y de Codenomicon, que lo descubrieron; y todas las organizaciones y usuarios que dependían de servidores con la versión vulnerable (OpenSSL 1.0.1 anterior a 1.0.1g).

Cómo se pudo arreglar. Bastaba con validar, antes de copiar, que la longitud que decía el cliente no fuera mayor que el tamaño real de los datos recibidos, y así fue como se corrigió. La versión arreglada, OpenSSL 1.0.1g, salió el 7 de abril de 2014. Después de eso, las organizaciones tuvieron que actualizar OpenSSL y, por precaución, cambiar sus claves y certificados. Llevado a C++, la lección es no confiar en tamaños que vienen de afuera y revisar siempre los límites, por ejemplo con vector::at() o comparando los índices con size().

Fuentes consultadas.

1. Chirgwin, R. y Williams, C. (2014). Anatomy of OpenSSL's Heartbleed: Just four bytes trigger horror bug. The Register.
2. OWASP. Heartbleed Bug.
3. NIST. Bugs Framework: CVE-2014-0160 Heartbleed, heap buffer over-read in OpenSSL.
4. NIST. National Vulnerability Database: CVE-2014-0160.

Caso 2. Cloudbleed (Cloudflare): el buffer que se desbordó y mezcló datos de otros sitios

Qué pasó. En febrero de 2017 se descubrió que los servidores de Cloudflare, una de las redes de distribución de contenido más grandes, estaban devolviendo en sus respuestas pedazos de memoria con datos privados de otros clientes. Al problema lo llamaron Cloudbleed, haciendo referencia a Heartbleed.

Por qué falló. Todo vino de un error de programación en un analizador de HTML. Cloudflare había empezado a escribir uno nuevo, llamado cf-html, para reemplazar uno viejo basado en Ragel que ya era muy difícil de mantener. La causa raíz fue que el final del buffer se revisaba con el operador de igualdad,

de modo que un puntero podía pasarse del final sin que la comprobación se diera cuenta. El mismo Cloudflare explicó que si hubieran usado >= en vez de ==, el desbordamiento se habría detectado. Es decir, una sola comparación mal escogida fue suficiente para que el programa leyera memoria que no le correspondía.

Personas, empresas y consecuencias. Estuvieron implicados Cloudflare, porque el error era de su código; Tavis Ormandy, investigador de Google Project Zero, que lo encontró mientras trabajaba en otro proyecto; y los clientes de Cloudflare cuyos datos se filtraron. Entre lo que quedó expuesto había cookies HTTP, tokens de autenticación y cuerpos de peticiones POST, y parte de esa información terminó guardada en la caché de los buscadores. Ormandy encontró datos de empresas como Uber, 1Password, FitBit y OKCupid. El analizador nuevo se había puesto en uso por primera vez el 22 de septiembre de 2016, pero según el informe de Cloudflare el mayor impacto fue entre el 13 y el 18 de febrero de 2017, cuando aproximadamente 1 de cada 3,3 millones de peticiones podía filtrar memoria (alrededor del 0,00003 %). Cloudflare recibió el aviso el 17 de febrero de 2017.

Cómo se pudo arreglar. Cloudflare reaccionó rápido, ya que desactivó tres funciones que usaban el mismo analizador (ofuscación de correos, Server-side Excludes y Automatic HTTPS Rewrites), corrigió la comprobación de límites (comparando con >= en lugar de ==) y trabajó para limpiar los datos que habían quedado en cachés. Lo que me deja este caso es que basta una comparación insuficiente para perder el control de un buffer, y que la mejor defensa es comprobar siempre que un índice o un puntero no se pase del tamaño reservado.

Fuentes consultadas.

1. Graham-Cumming, J. (2017). Incident report on memory leak caused by Cloudflare parser bug. Cloudflare Blog.
2. The Register (2017). Cloudbleed: Big web brands 'leaked crypto keys, personal secrets' thanks to Cloudflare bug.
3. The Hacker News (2017). Cloudflare vulnerability exposes sensitive data.
4. GovInfoSecurity (2017). 'Cloudbleed' Leaked Data from Uber, 1Password, FitBit, OKCupid and More.

Caso 3. Operación Aurora (Internet Explorer): el puntero que apuntaba a un objeto ya eliminado

Qué pasó. Entre finales de 2009 y comienzos de 2010 hubo una serie de ataques dirigidos contra grandes empresas que se conoce como Operación Aurora. La puerta de entrada fue una vulnerabilidad que nadie conocía hasta ese momento (un zero-day) en Internet Explorer, registrada como CVE-2010-0249, y que se explotó en la práctica en diciembre de 2009 y enero de 2010.

Por qué falló. Era un fallo de tipo use-after-free, en el que un atacante remoto podía ejecutar código arbitrario accediendo a un puntero de un objeto que ya había sido eliminado. Dicho de otra forma, el navegador liberaba la memoria de un objeto pero se quedaba con un puntero colgante que después seguía usando. Como esa memoria podía haberse reasignado a otra cosa, el atacante armaba una página web que llenaba ese espacio con sus propios datos, y así lograba ejecutar su código. La vulnerabilidad afectaba a Internet Explorer 6, 6 SP1, 7 y 8 en varias versiones de Windows.

Personas, empresas y consecuencias. Estuvieron implicados Microsoft, porque el error estaba en su navegador; las empresas atacadas; y los usuarios de las versiones vulnerables de Internet Explorer. Según

los reportes de esa época, entre las víctimas estuvieron Google, Juniper y Adobe, junto con decenas de empresas más, y Microsoft dijo que solo había visto ataques limitados contra Internet Explorer 6. Para caer bastaba con que la víctima visitara una página manipulada, y el resultado fue el robo de propiedad intelectual desde redes corporativas.

Cómo se pudo arreglar. Desde el código, la solución es nunca usar un puntero después de liberar el objeto, ya sea asignándole nullptr después del delete, dejando claro quién es el dueño de cada objeto o usando punteros inteligentes de C++ (std::unique_ptr, std::shared_ptr), que liberan la memoria solos y evitan los punteros colgantes. Del lado del producto, Microsoft publicó el aviso de seguridad 979352 y corrigió la vulnerabilidad con una actualización.

Fuentes consultadas.

1. NIST. National Vulnerability Database: CVE-2010-0249.
2. Microsoft. Security Advisory 979352: Vulnerability in Internet Explorer Could Allow Remote Code Execution.

5. Cómo fue el desarrollo

5.1 Problemas que tuvimos y cómo los resolvimos

Problema	Por qué pasó	Qué hicimos
OnlineGDB mostraba "file format not recognized; treating as linker script" al compilar.	Tenía archivos .csv en el proyecto y OnlineGDB intenta enlazar todos los archivos, así que trataba el .csv como si fuera un script.	Les cambié la extensión a .txt (ice_cream.txt y student_scores.txt). El compilador los ignora y C++ los lee igual.
El archivo de helados mostraba números absurdos, como −4.662.262.677.220.200.	La versión V1 del CSV estaba dañada. Lo más probable es que al abrirla y guardarla con otra configuración regional, los puntos decimales se volvieron separadores de miles.	Descarté esa versión y usé el archivo original de EAFIT Interactiva, sin abrirlo ni guardarlo en Excel.
Error de compilación "expected '}' at end of input".	Al pegar todo el código en OnlineGDB, el texto se cortó antes del final del archivo.	Pegué el código por partes y revisé que el archivo terminara con return 0; y la llave que cierra main().
La regresión múltiple decía "Variables: 0" y daba un R^2 negativo.	El separador del archivo de estudiantes no era ; (seguramente cambió al copiar el contenido). Como stod lee el número del inicio de la línea e ignora el resto sin avisar, cada fila quedaba con una sola columna.	Reescribí loadCSV para que detecte el separador (;, coma o tabulador) y acepte decimales con coma, y agregué un mensaje de error para cuando no se detecten columnas de entrada.

Problema	Por qué pasó	Qué hicimos
La regresión simple de helados dio un R^2 de apenas 0.08.	La relación entre temperatura y ventas no es lineal, sino una parábola en U.	Lo dejé documentado y agregué una prueba extra con x^2, con la que el R^2 sube a 0.88 usando la misma clase.
Riesgo de sesgo al evaluar el modelo.	El archivo de helados viene ordenado por temperatura, y además el escalado del test tiene que hacerse con las estadísticas del entrenamiento.	Dividí los datos tomando una de cada cinco filas para prueba, y le puse a data_scaling el parámetro fitScaler para reutilizar la media y la desviación del entrenamiento.
El programa se cerraba con error de segmentación o con una excepción de stod cuando los datos estaban mal.	No estaba validando las entradas (matrices vacías, tamaños distintos, escalar el test sin haber entrenado, predecir con otras dimensiones o archivos con texto).	Agregué validaciones con excepciones y mensajes claros (checkMatrix, parseNumber y controles en cada método), las atrapo en main() con try/catch y documenté nueve casos en pruebaValidaciones().

5.2 Uso de Inteligencia Artificial

Durante la práctica usé el asistente de IA Claude (Anthropic) como un tutor que me acompañó en el proceso. Lo usé de estas formas:

- Para diagnosticar errores de compilación y comportamientos raros, como los .csv tratados como scripts, el separador incorrecto en el archivo de estudiantes y el archivo de helados dañado.
- Para buscar noticias y fuentes acerca de los fallos históricos para que tuviera relevancia con lo que se pedía en la práctica
- Para mejorar la redacción del informe, de modo que las ideas quedaran bien conectadas y las secciones fueran coherentes entre sí.
- Para organizar las tareas de la práctica en pasos ordenados e ir avanzando uno a la vez.
- Para repasar temas y conceptos (regresión lineal, estandarización, gradiente descendente y memoria global, local y dinámica) antes de implementar cada parte.
- Para revisar el código contra lo que pedía el enunciado y reforzarlo con métodos auxiliares de matrices y estadística y con validación de entradas.

6. Conclusiones

1. Estandarizar los datos fue necesario para que el gradiente descendente convergiera con una tasa de aprendizaje razonable, sobre todo cuando las variables tienen escalas muy distintas (por ejemplo, horas de estudio entre 1 y 10 contra asistencia entre 50 y 100). También aprendí que la media y la desviación se deben calcular solo con los datos de entrenamiento y reutilizarse en el test, y por eso data_scaling guarda esos valores dentro de la clase.

2. Gracias al diseño orientado a objetos pude usar la misma clase LinearRegression, sin modificarla, en tres problemas distintos, que fueron la regresión simple (una variable), la regresión múltiple (cuatro variables) y la regresión con una variable no lineal agregada (x^2). Encapsular los atributos y separar las tareas en métodos (fit, predict, score y data_scaling) me permitió probar cada responsabilidad por aparte.
3. El resultado de un modelo depende mucho de la forma de los datos. En el conjunto de helados la relación es una parábola, y por eso el modelo lineal solo llegó a un R^2 de 0.08, mientras que al agregar x^2 subió a 0.88. En cambio, en el conjunto de estudiantes la relación es básicamente lineal y el R^2 fue de 0.85. Esto me enseñó que una métrica baja no siempre significa que el código esté mal, sino que el modelo puede no corresponder a los datos.
4. En la regresión múltiple, tener los pesos en escala estandarizada me permitió comparar qué tan importante es cada variable. Las horas estudiadas (4.98) pesan más que las calificaciones previas (2.65), la asistencia (1.59) y las horas de sueño (1.17), lo que coincide con las correlaciones de los datos.
5. Con el análisis de memoria vi que sizeof depende de la clase y no del lugar donde se crea el objeto (96 bytes en global, local y dinámico). También vi que el relleno (padding) puede hacer que el tamaño sea mayor que la suma de los miembros (96 frente a 92), y que un vector siempre mide 24 bytes porque sus datos están aparte en el heap (480 bytes para cinco elementos de 96 bytes).
6. Los tres fallos históricos que investigué (Heartbleed, Cloudbleed y Operación Aurora) tienen algo en común, ya que todos vienen de no validar los límites de un buffer o de usar memoria que ya se había liberado. En C++ moderno estos riesgos se reducen usando contenedores como std::vector (que libera su memoria solo), accediendo con comprobación de límites (at()) y usando punteros inteligentes en vez de new y delete manuales. Mi clase sigue esa idea, porque solo uso new en el análisis de memoria y cada uno de esos objetos lo libero con delete (o con delete[] en el caso del arreglo).
7. Muchas de las dificultades que tuve no fueron de lógica sino de datos y de entorno, como un CSV dañado, un separador distinto al que esperaba y archivos que el compilador en línea trataba como scripts. Por eso me quedó claro que validar los datos de entrada (por ejemplo, revisando cuántas columnas se leyeron) es tan importante como el algoritmo.
8. Separar las operaciones de estadística y de matrices en funciones auxiliares (MathUtils) y validar las entradas con excepciones hizo que el código fuera más fácil de leer, de probar y más robusto. Ahora, si llegan datos inválidos, el programa avisa con un mensaje claro en vez de cerrarse, y las nueve pruebas de validación dejan documentado ese comportamiento.